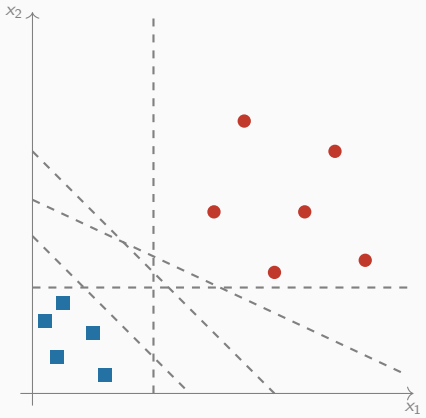
Support Vector Machines

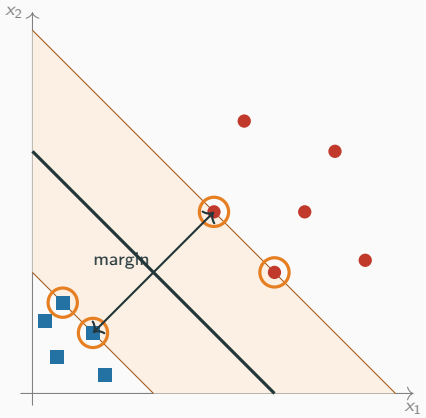
CSS 324: Introduction to Machine Learning

Linear separability



- A **linear classifier** separates two classes with a line. In n dimensions the line becomes a **hyperplane**.
- The data are **linearly separable** if at least one such line separates the classes perfectly.
- In that case there are *infinitely many* separating lines (gray dashed). Each one classifies the training set perfectly.
- Question: which separating line should we choose?

The SVM idea: the widest street



- A line close to the training points is fragile: a small change in the data can flip a prediction.
- SVM idea: choose the line that stays *as far as possible* from the closest points of both classes. That line has the widest "street" (**margin**) between the classes.
- The closest points (circled) are the **support vectors**. They alone determine the line.
- This approach is called **large margin classification**.

Decision function and prediction

For an instance $\mathbf{x} = (x_1, \ldots, x_n)^\top \in \mathbb{R}^n$, a linear SVM with weight vector $\mathbf{w} \in \mathbb{R}^n$ and bias $b \in \mathbb{R}$ computes the **decision function**

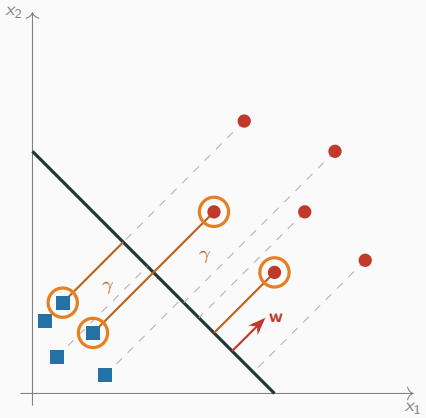
$$h(\mathbf{x}) \;=\; \mathbf{w}^\top \mathbf{x} + b \;=\; w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b.$$

Linear SVM classifier prediction

$$\hat{y} \;=\; \begin{cases} 0 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0, \\ 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0. \end{cases}$$

- A nonnegative result predicts the positive class (1). A negative result predicts the negative class (0).
- The **decision boundary** $\{\mathbf{x} \in \mathbb{R}^n : \mathbf{w}^\top \mathbf{x} + b = 0\}$ is a line when $n = 2$ and a hyperplane in general.
- **Training** a linear SVM means finding the $\mathbf{w}$ and b that give the widest margin.

The margin



Distance from a point to the hyperplane

$$d_i = \frac{|\mathbf{w}^\top \mathbf{x}_i + b|}{\|\mathbf{w}\|}$$

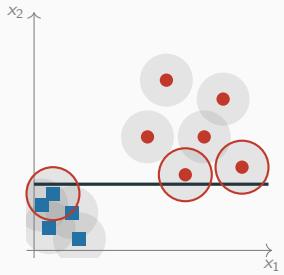
Margin of a separating hyperplane

$$\gamma(\mathbf{w}, b) = \min_{i=1,\ldots,m} \frac{|\mathbf{w}^\top \mathbf{x}_i + b|}{\|\mathbf{w}\|}$$

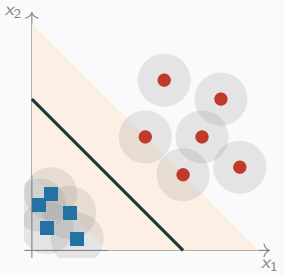
- The margin is the distance to the *closest* training point. The width of the street is 2γ.
- $\mathbf{w}$ is perpendicular to the boundary. Dividing by $\|\mathbf{w}\|$ turns $\mathbf{w}^\top \mathbf{x} + b$ into a distance.
- SVM: among all separating hyperplanes, choose the one with the **largest** γ.

Why a large margin? Robustness to noise

Small margin



Large margin



- New data are noisy versions of the training data. The disks show a perturbation of radius r around each point.
- **Left:** the boundary cuts through several disks (red), so a small perturbation of those points flips their predicted class.
- **Right:** every disk lies on one side. A perturbation smaller than γ *never* changes the prediction.

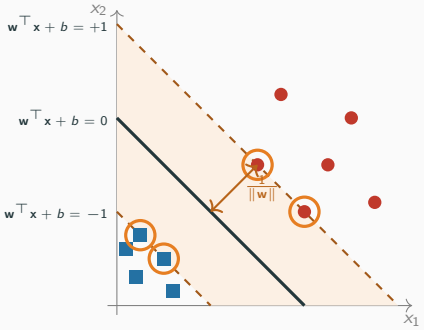
Why is just a separating line not enough?

1. **Training error cannot decide.** Every separating line has zero training error, yet they disagree on new points. For example, $\mathbf{x} = (2.5, 1.2)$ is positive for the line $x_1 = 2$ but negative for $x_1 + x_2 = 4$. We need an extra criterion.
2. **Robustness.** A line close to training points is sensitive to noise and measurement errors. A large margin gives a "safety zone" of radius γ around every training point.
3. **Better generalization.** Few lines fit through a wide street, so large-margin classifiers are a *simpler* model class that overfits less.
4. **A unique, well-defined answer.** The maximum-margin line is unique and depends only on the support vectors. Points far from the street do not affect it.

Bottom line

Separating the training data is the *minimum* requirement. Among all separating lines, the SVM chooses the most *confident* one: the line with the largest margin.

From maximum margin to an optimization problem



Encode the labels as $t^{(i)} = +1$ if $y^{(i)} = 1$ and $t^{(i)} = -1$ if $y^{(i)} = 0$. Then instance i is classified correctly iff $t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) > 0$.

Step 1: goal. Maximize the margin

$$\max_{\mathbf{w}, b} \min_{i} \frac{t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|}.$$

Step 2: fix the scale. $(\mathbf{w}, b)$ and $(c\mathbf{w}, cb)$ with $c > 0$ give the *same* line. So we may choose the scale such that the closest points satisfy $t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) = 1$. Then

$$\gamma = \frac{1}{\|\mathbf{w}\|}, \qquad \text{street width} = \frac{2}{\|\mathbf{w}\|}.$$

Step 3: simplify. Maximizing $\frac{1}{\|\mathbf{w}\|}$ is the same as minimizing $\frac{1}{2}\|\mathbf{w}\|^2 = \frac{1}{2}\mathbf{w}^\top \mathbf{w}$, which is smooth and convex.

Hard margin linear SVM

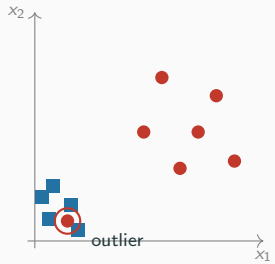
Hard margin linear SVM classifier objective

$$\underset{\mathbf{w},\, b}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^\top \mathbf{w} \qquad \text{subject to} \qquad t^{(i)}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b\right) \geq 1 \quad \text{for } i = 1, 2, \ldots, m$$

- **Objective** $\frac{1}{2}\mathbf{w}^\top \mathbf{w}$: a small $\|\mathbf{w}\|$ means a *wide* street, since the width is $2/\|\mathbf{w}\|$.
- **Constraints**: every instance lies on the correct side *and outside the street*. "Hard" means no violations are allowed.
- **Support vectors** are the instances with $t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) = 1$. They sit exactly on the edges of the street.
- This is a **convex quadratic program (QP)**: a quadratic objective with linear constraints. For linearly separable data it has a *unique* solution, which standard QP solvers can find.
- Limitation: if the data are **not** linearly separable, the problem has no solution. A single outlier can also make the margin very narrow.

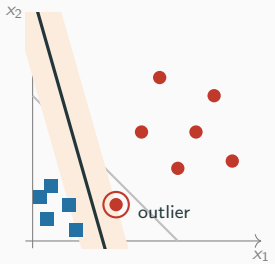
Soft margin classification: why?

Not linearly separable



No line separates the classes, so the hard margin problem has *no solution*.

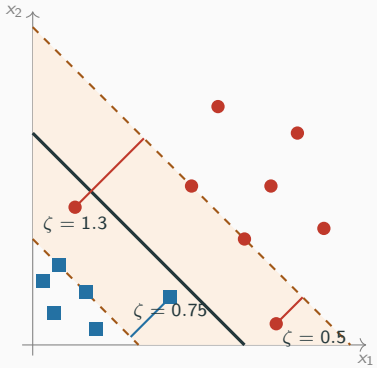
Outlier squeezes the margin



The hard margin solution exists but is very narrow. It will likely generalize poorly.

- The hard margin SVM is **sensitive to outliers** and works **only** for linearly separable data.
- Soft margin idea: find a good balance between keeping the street *as wide as possible* and limiting the **margin violations**, i.e. instances that end up inside the street or even on the wrong side.

Slack variables



For each instance introduce a **slack variable** $\zeta^{(i)} \geq 0$ that measures how much it violates the margin, and relax the constraint to

$$t^{(i)}\big(\mathbf{w}^\top \mathbf{x}^{(i)} + b\big) \;\geq\; 1 - \zeta^{(i)}.$$

- $\zeta^{(i)} = 0$: on or outside the street, correct side. No violation.
- $0 < \zeta^{(i)} \leq 1$: inside the street but still correctly classified.
- $\zeta^{(i)} > 1$: on the wrong side of the boundary, i.e. **misclassified**.

The smallest valid slack is

$$\zeta^{(i)} = \max\big(0,\; 1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)\big).$$

Soft margin linear SVM

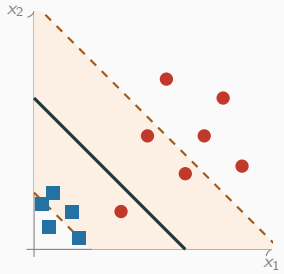
Soft margin linear SVM classifier objective

$$\underset{\mathbf{w},\, b,\, \zeta}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^\top \mathbf{w} \;+\; C\sum_{i=1}^{m}\zeta^{(i)} \qquad \text{subject to} \quad t^{(i)}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b\right) \geq 1 - \zeta^{(i)} \quad \forall i$$
$$\zeta^{(i)} \geq 0$$

- Two **conflicting objectives**:
  - $\frac{1}{2}\mathbf{w}^\top \mathbf{w}$ small $\Rightarrow$ wide street;
  - $\sum_i \zeta^{(i)}$ small $\Rightarrow$ few and small margin violations.
- The hyperparameter $C > 0$ sets the trade-off between them.
- The problem is still a **convex QP**, and it *always* has a solution (take $\zeta^{(i)}$ large enough), even for data that are not linearly separable.
- As $C \to \infty$, violations become infinitely expensive and we recover the **hard margin** SVM.

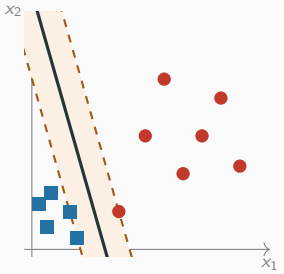
The hyperparameter C

Small C (e.g. $C = 0.1$)



Wide street, more margin violations.
Lower variance, may *underfit*.

Large C (e.g. $C = 100$)



Narrow street, few or no violations.
Fits the outlier, may *overfit*.

- The model with small C is likely to generalize better here, even though it makes more violations on the training set.
- Rule of thumb: if your SVM model is overfitting, try **reducing** C. Tune C with cross-validation.

Equivalent view: the hinge loss



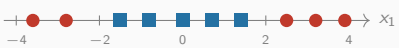
Substituting the optimal slack $\zeta^{(i)} = \max\left(0, 1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)\right)$ gives an **unconstrained** problem:

$$J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^\top \mathbf{w} \; + \; C \sum_{i=1}^{m} \max\Big(0, \; 1 - t^{(i)}\big(\mathbf{w}^\top \mathbf{x}^{(i)} + b\big)\Big).$$

- With $s = t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$, the loss is 0 for instances off the street on the correct side ($s \geq 1$), and grows *linearly* with the violation.
- The hinge loss is a convex upper bound on the 0–1 (misclassification) loss.
- J can be minimized by (sub)gradient descent, e.g. `SGDClassifier(loss="hinge")` in Scikit-Learn.

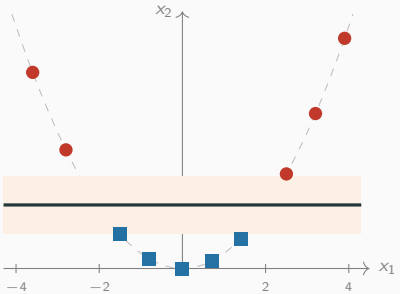
Nonlinear data: adding features

Original feature x_1



No single threshold on x_1 separates the classes.

Add feature $x_2 = x_1^2$



Now the data are **linearly separable**.

- **Idea:** map each instance to a higher-dimensional feature space, $\mathbf{x} \mapsto \phi(\mathbf{x})$, and train a *linear* SVM there. A line in feature space is a *curve* in the original space.
- In Scikit-Learn: `PolynomialFeatures(degree=3)` followed by `LinearSVC`.

The problem with explicit features

- Polynomial features of degree d in n original features give $\binom{n+d}{d}$ new features:

n	degree d	number of features
2	3	10
10	3	286
100	3	176,851
1000	3	$\approx 1.7 \times 10^8$

- A low degree cannot fit complex data. A high degree creates a **combinatorial explosion** of features, which makes the model too slow to train and to use.
- For some useful feature maps (e.g. the Gaussian RBF below), $\phi(\mathbf{x})$ is even **infinite-dimensional**.

Question: can we get the benefit of a high-dimensional ϕ *without* ever computing $\phi(\mathbf{x})$? **Yes: the kernel trick.**

The dual problem: data only through dot products

Using Lagrange duality, the soft margin SVM can be rewritten as an equivalent **dual problem** in variables $\alpha^{(i)}$. We state it without derivation:

Dual form of the linear SVM objective

$$\underset{\boldsymbol{\alpha}}{\text{minimize}} \quad \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\underbrace{\mathbf{x}^{(i)\top}\mathbf{x}^{(j)}}_{\text{dot product}} - \sum_{i=1}^{m}\alpha^{(i)} \qquad \text{subject to} \quad \begin{array}{l} 0 \le \alpha^{(i)} \le C \\ \sum_i \alpha^{(i)}t^{(i)} = 0 \end{array}$$

From the solution $\hat{\boldsymbol{\alpha}}$:

$$\hat{\mathbf{w}} = \sum_{i=1}^{m}\hat{\alpha}^{(i)}t^{(i)}\mathbf{x}^{(i)}, \qquad h(\mathbf{x}) = \hat{\mathbf{w}}^{\top}\mathbf{x} + \hat{b} = \sum_{\hat{\alpha}^{(i)}>0}\hat{\alpha}^{(i)}t^{(i)}\,\mathbf{x}^{(i)\top}\mathbf{x} + \hat{b}.$$

- $\hat{\alpha}^{(i)} > 0$ *only* for the support vectors, so the sum is over a few instances.
- Key observation: both training and prediction use the data **only through dot products** $\mathbf{x}^{(i)\top}\mathbf{x}^{(j)}$.

The kernel trick

Example. The 2nd-degree polynomial map for $\mathbf{x} = (x_1, x_2)$:

$$\phi(\mathbf{x}) = \left(x_1^2,\ \sqrt{2}\, x_1 x_2,\ x_2^2\right)^\top \in \mathbb{R}^3.$$

For two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^2$:

$$\phi(\mathbf{a})^\top \phi(\mathbf{b}) = a_1^2 b_1^2 + 2 a_1 b_1 a_2 b_2 + a_2^2 b_2^2 = (a_1 b_1 + a_2 b_2)^2 = \left(\mathbf{a}^\top \mathbf{b}\right)^2.$$

The dot product in feature space is computed **in the original space**, without computing ϕ.

Kernel

A **kernel** is a function $K(\mathbf{a}, \mathbf{b}) = \phi(\mathbf{a})^\top \phi(\mathbf{b})$ that computes the dot product of the transformed vectors from $\mathbf{a}$ and $\mathbf{b}$ alone.

- Kernel trick: in the dual problem and in the prediction, replace every ${\mathbf{x}^{(i)}}^\top \mathbf{x}^{(j)}$ by $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$.
- The result is the same as training a linear SVM on $\phi(\mathbf{x})$, but we never compute or store $\phi(\mathbf{x})$.

Common kernels

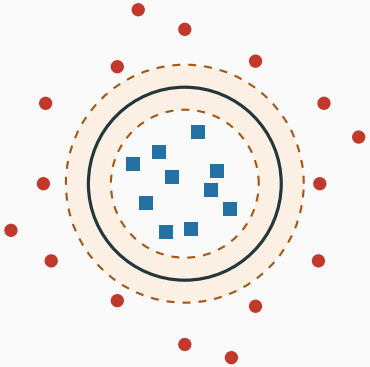
Kernel	$K(\mathbf{a}, \mathbf{b})$	Hyperparameters
Linear	$\mathbf{a}^\top \mathbf{b}$	—
Polynomial	$\left(\gamma\, \mathbf{a}^\top \mathbf{b} + r\right)^d$	degree d, γ, r
Gaussian RBF	$\exp\left(-\gamma \lVert \mathbf{a} - \mathbf{b} \rVert^2\right)$	$\gamma > 0$
Sigmoid	$\tanh\left(\gamma\, \mathbf{a}^\top \mathbf{b} + r\right)$	γ, r

Mercer's theorem (informal)

If K is **symmetric** and, for every finite set of points $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}$, the **Gram matrix** $\left[K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})\right]_{i,j}$ is **positive semidefinite**, then there exists a map ϕ (possibly into an infinite-dimensional space) such that $K(\mathbf{a}, \mathbf{b}) = \phi(\mathbf{a})^\top \phi(\mathbf{b})$.

- We do not need to know ϕ, only that it exists. For the Gaussian RBF kernel, ϕ is infinite-dimensional.
- The sigmoid kernel violates Mercer's conditions for some γ, r, but often works in practice.

Nonlinear SVM: kernelized prediction



RBF kernel: a circular boundary in the original space.

Training solves the dual with K, giving $\hat{\alpha}^{(i)}$. A new instance $\mathbf{x}$ is classified using **only the support vectors**:

$$h(\mathbf{x}) = \sum_{\hat{\alpha}^{(i)}>0} \hat{\alpha}^{(i)} t^{(i)} K\left(\mathbf{x}^{(i)}, \mathbf{x}\right) + \hat{b},$$

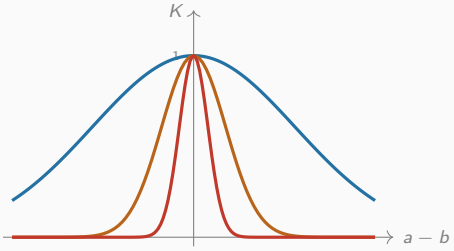
$$\hat{y} = \begin{cases} 0 & h(\mathbf{x}) < 0 \\ 1 & h(\mathbf{x}) \geq 0 \end{cases}$$

with the bias computed from the support vectors:

$$\hat{b} = \frac{1}{n_s} \sum_{\hat{\alpha}^{(i)}>0} \left(t^{(i)} - \sum_{\hat{\alpha}^{(j)}>0} \hat{\alpha}^{(j)} t^{(j)} K\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right) \right).$$

- The decision boundary is **nonlinear** in the original space.
- $\hat{\mathbf{w}}$ is never computed, since it lives in feature space.

The Gaussian RBF kernel and the hyperparameter γ



1-D example: $K(a, b) = \exp(-\gamma(a - b)^2)$

$\gamma = 0.1$, $\gamma = 1$, $\gamma = 5$

- $K(\mathbf{a}, \mathbf{b})$ is a **similarity**: 1 when $\mathbf{a} = \mathbf{b}$, decaying to 0 as the points move apart.
- $h(\mathbf{x})$ is a weighted sum of "bumps" centered at the support vectors.
- **Small γ:** wide bumps, so each instance has a large range of influence. The boundary is *smooth* and the model may *underfit*.
- **Large γ:** narrow bumps. The boundary is *irregular* and wiggles around individual instances, so the model may *overfit*.
- γ acts like a regularization hyperparameter: if the model overfits, **reduce** γ. If it underfits, increase it. Tune γ together with C.

Which kernel and which class to use?

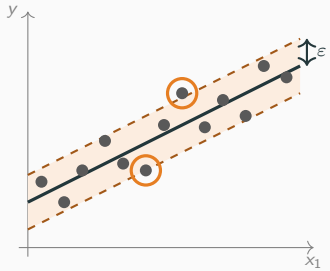
- Always try the **linear** kernel first. `LinearSVC` is much faster than `SVC(kernel="linear")`, especially for large training sets or many features.
- If the training set is not too large, try the **Gaussian RBF** kernel next. It works well in most cases.
- Use **cross-validation and grid search** to choose the kernel and to tune C, γ (and d).
- SVMs are sensitive to feature scales, so always **scale the features** first (e.g. `StandardScaler`).

Class	Time complexity	Out-of-core	Scaling required	Kernel trick
`LinearSVC`	$O(m \times n)$	No	Yes	No
`SGDClassifier`	$O(m \times n)$	Yes	Yes	No
`SVC`	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes

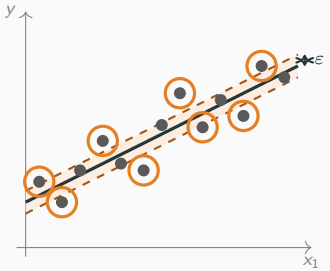
m = number of instances, n = number of features. `SVC` becomes very slow for large m (hundreds of thousands of instances).

SVM regression: reversing the objective

Large ε ($\varepsilon = 0.6$)



Small ε ($\varepsilon = 0.25$)



- **Classification:** fit the widest street with *no* instances inside it.
- Regression: fit a street of width 2ε with as many instances as possible *inside* it, while limiting the instances that fall outside.
- The model is ε-**insensitive**: adding training instances inside the street does not change the predictions. Only instances on or outside the street (circled) are **support vectors**.

Linear SVM regression: the optimization problem

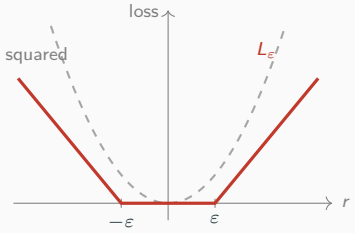
With prediction $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ and real-valued targets $y^{(i)}$:

Linear SVR objective

$$\begin{aligned} \underset{\mathbf{w}, b, \boldsymbol{\zeta}, \boldsymbol{\zeta}^*}{\text{minimize}} \quad & \frac{1}{2}\mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^{m} \left(\zeta^{(i)} + \zeta^{*(i)}\right) \\ & y^{(i)} - h(\mathbf{x}^{(i)}) \leq \varepsilon + \zeta^{(i)} \quad \text{(above the street)} \\ \text{s.t.} \quad & h(\mathbf{x}^{(i)}) - y^{(i)} \leq \varepsilon + \zeta^{*(i)} \quad \text{(below the street)} \\ & \zeta^{(i)},\ \zeta^{*(i)} \geq 0. \end{aligned}$$

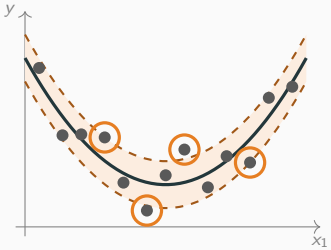
Equivalently, minimize $\frac{1}{2}\mathbf{w}^\top \mathbf{w} + C \sum_i L_\varepsilon\left(y^{(i)} - h(\mathbf{x}^{(i)})\right)$ with the ε-**insensitive loss**

$$L_\varepsilon(r) = \max\left(0,\ |r| - \varepsilon\right).$$



Errors smaller than ε cost nothing. Larger errors cost *linearly*, so the model is robust to outliers.

Nonlinear SVM regression



2nd-degree polynomial kernel, $\varepsilon = 0.5$.

Apply the **kernel trick** to the dual of the SVR problem. The prediction becomes

$$h(\mathbf{x}) = \sum_{i \in \mathrm{SV}} \left(\hat{\alpha}^{(i)} - \hat{\alpha}^{*(i)}\right) K\left(\mathbf{x}^{(i)}, \mathbf{x}\right) + \hat{b}.$$

Hyperparameters:

- ε: width of the street. A larger ε gives fewer support vectors and a simpler model.
- C: penalty for instances outside the street. A smaller C gives more regularization.
- Kernel and its parameters (d, γ, ...).

Scikit-Learn:

```
LinearSVR(epsilon=0.5)
SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
```

Scaling and complexity: as for `LinearSVC` and `SVC`.

Summary

- **Large margin classification:** among all separating hyperplanes, choose the one with the widest street. The street is determined only by the **support vectors**.
- **Hard margin:** $\min \frac{1}{2}\mathbf{w}^\top\mathbf{w}$ s.t. $t^{(i)}(\mathbf{w}^\top\mathbf{x}^{(i)} + b) \geq 1$. Requires linearly separable data and is sensitive to outliers.
- **Soft margin:** add slack variables $\zeta^{(i)}$ and the penalty $C\sum_i \zeta^{(i)}$, which is equivalent to the hinge loss. C trades street width against violations: if the model overfits, reduce C.
- **Kernel trick:** the dual uses data only through dot products. Replacing them by $K(\mathbf{a}, \mathbf{b}) = \phi(\mathbf{a})^\top\phi(\mathbf{b})$ gives nonlinear boundaries without computing ϕ.
- **Kernels:** linear, polynomial, Gaussian RBF (tune γ: if overfitting, reduce it), sigmoid.
- **SVM regression:** fit as many instances as possible *inside* a street of width 2ε, using the ε-insensitive loss.

In practice

Scale the features $\rightarrow$ try `LinearSVC`/`LinearSVR` first $\rightarrow$ then `SVC`/`SVR` with an RBF kernel $\rightarrow$ tune C, γ (and ε) with cross-validation.